

Clueless Coders

SOFTWARE DESIGN DOCUMENT

Prepared by: Ezra Beletskiy | Chris Dixon | Evan Edelstein | Santino Nardolillo | Dan Xu

Date: April 7th, 2025

Table of Contents

Introduction	2
Overview	2
Scope	2
System Architecture	3
Design Diagram	3
Class Descriptions	4
Server:	4
Game:	4
Player:	6
Packet:	7
GamePacket:	7
SocketHandler:	8
Client:	9
Gameboard:	9
Room:	10
DeckHelper:	10
Deck:	11
Card:	11
System Operations	12

Introduction

Overview

The **Software Design Document (SDD)** for the Clue-less game outlines the technical design and architecture necessary to implement a multiplayer, network-based version of the Clue board game. This document focuses on the structural and behavioral design of the system, ensuring that it can efficiently support real-time interactions between players in different locations. It highlights the essential components of the system, including the client-server architecture, entity classes, and key interactions between those components during gameplay.

This document provides a detailed breakdown of the **class architecture** through **class diagrams**, **class descriptions**, and their relationships, outlining the core components such as the **Server**, **Player**, **Game**, **Packet**, and **SocketHandler**. Each entity class is defined with its attributes, methods, and responsibilities, ensuring a clear understanding of the modular structure of the system.

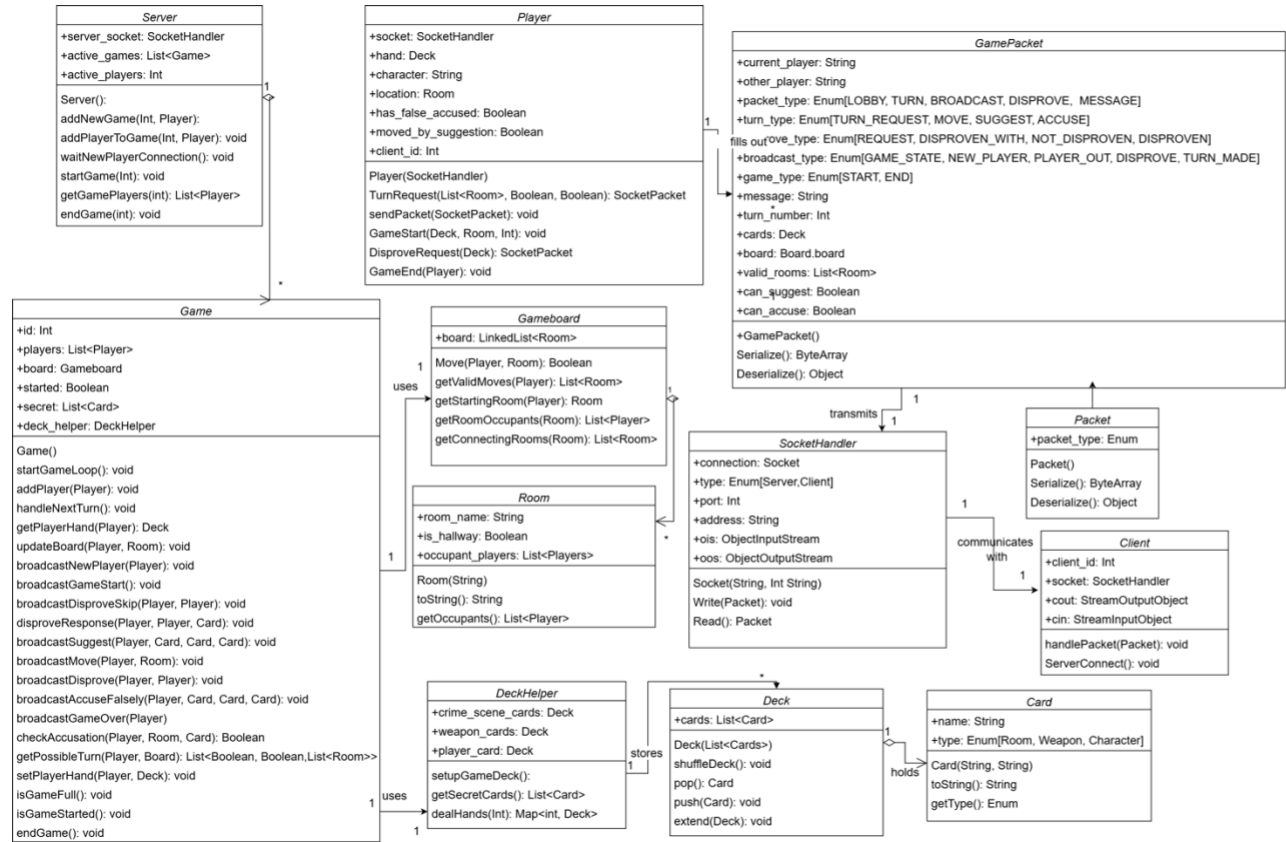
Additionally, the dynamic aspects of the system are described through interaction models, such as sequence diagrams, which document the flow of key actions, such as making suggestions, disproving suggestions, and game state updates. The design described in this document serves as the foundation for implementing game logic, user interactions, and networking functionality while adhering to the principles of modularity and scalability.

Scope

The software design document aims to lay the foundation for the implementation of Clue-less. This document focuses on the architecture and backend of the game. Further documentation will focus on the user interface and user experience.

System Architecture

Design Diagram



Class Descriptions

Server:

Attribute	Description
server_socket: SocketHandler	A SocketHandler class used to connect new clients
active_games: List<Game>	A list of active Game objects
active_players: Int	Number of active players

Method	Description
Server()	constructor
startGame(Game): void	Start the game loop
waitNewPlayerConnection(): void	Poll for new client connections. When a player connects increment the player counter
addNewGame(Int, Player)	Add a new game with a specified id and player
addPlayerToGame(Int, Player)	Add a player to the game matching the id provided.
getGamePlayers(int): List<Player>	Lookup the players associated with a game id
endGame(int): void	Remove the game from the list of active games

Game:

Attribute	Description
id: Int	A UUID used to identify the game
players: List<Player>	A list of active players in the game
board: Gameboard	The board object used to track player location
secret: List<Card>	The secret weapon, crime scene and character that each player is trying to deduce

deck_helper: DeckHelper	DeckHelper object used to manage the cards in the game
Started: Boolean	Has the game started. New players cannot be added to a started game

Method	Description
Game()	constructor
startGameLoop(): void	Begin playing the game
addPlayer(Player): void	Add player to game
handleNextTurn(): void	Initiate the turn sequence for the next player
getPlayerHand(Player): deck	Get the cards held by player
setPlayerHand(Player, deck): void	Set the hand of a player
updateBoard(Player, Room): void	Move a player to the specified room
broadcastNewPlayer(Player): void	Notify all players a new player has join
broadcastGameStart(): void	Notify all players the game is starting
broadcastDisproveSkip(Player, Player): void	Notify all players a player could not disprove a suggestion
disproveResponse(Player, Player, Card): void	Notify a player whom disproved their suggestion and with what card
broadcastSuggest(Player, Card, Card, Card): void	Notify all players of a suggestion or accusation
broadcastMove(Player, Room): void	Notify all players of a players movement
broadcastDisprove(Player, Player): void	Notify all players that a disproval has been made
broadcastAccuseFalsely(Player, Card, Card, Card): void	Notify all players that an accusation wrong
broadcastGameOver(Player)	Notify all players that the game is over
checkAccusation(Player, Room, Card): Boolean	Check the validity of an accusation
getPossibleTurn(Player):List<Boolean,Boolean, List<Room>>	Get possible turn and available moves for a player

isGameFull(): void	Are the maximum number of players in the game
isGameStarted(): void	Has the game started
endGame(): void	Inform the server that the game has ended

Player:

Attribute	Description
socket: SocketHandler	A socket object used to communicate with the client
hand: Deck	The cards the player holds
character: String	The character name
location: Room	The location of the player
has_accused_falsey: Boolean	If the player is out after making a false accusation
moved_by_suggestion: Boolean	Was the player moved to current room by a suggestion
client_id: Int	The UUID of the client

Method	Description
Player(SocketHandler)	Constructor
turnRequest(List<Room>, Boolean, Boolean): SocketPacket	Send a request message to the client and await a turn response
sendPacket(SocketPacket): void	Send packet to the client
disproveRequest(Deck): SocketPacket	Send a disprove request message to the client and await a disprove response
gameStart(Deck, Room, Int)	Send a message to the client signifying the game is starting
gameEnd(Player): void	Send a message to the client signifying the game has ended and the player has won

Packet:

Attribute	Description
packet_type: Enum	The type of the packet

Method	Description
Packet()	constructor
Serialize(): ByteArray	Convert the object into a raw binary array
Deserialize(ByteArray): Object	Convert a binary array into the packet object

GamePacket:

Attribute	Description
packet_type: Enum[LOBBY, TURN, BROADCAST, DISPROVE, MESSAGE]	The type of the packet
current_player: String	The character name sending the packet
other_player: String	The subject character if the packet
turn_type: Enum[TURN_REQUEST, MOVE, SUGGEST, ACCUSE]	The turn type of the packet
disprove_type: Enum[REQUEST, DISPROVEN_WITH, NOT_DISPROVEN, DISPROVEN]	The disprove type of the packet
broadcast_type: Enum[GAME_STATE, NEW_PLAYER, PLAYER_OUT, DISPROVE, TURN_MADE]	The type of broadcast this packet represents
game_type: Enum[START, END]	The type of game state update
message: String	A message to pass between the server and client
turn_number: Int	The turn number of the game
board: Board.board	Player locations
valid_rooms: List<Room>	Valid rooms the player can move too

can_suggest: Boolean	Can the client suggest on their turn
can_accuse: Boolean	Can the client accuse on their turn

Method	Description
GamePacket()	constructor
Serialize(): ByteArray	Convert the object into a raw binary array
Deserialize(ByteArray): Object	Convert a binary array into the packet object

SocketHandler:

Attribute	Description
connection: Socket	A TCP socket used to communicate between server and client
type: Enum[Server,Client]	If the socket represents the client or server
port: Int	The socket port
Address: String	The socket address
ois: ObjectInputStream	Stream object to send bytes over the connection
oos: ObjectOutputStream	Stream object to receive data over the connection

Method	Description
SocketHandler	constructor
Write(Packet): void	Send the packet over the connection
Read(): Packet	Read data from the oos and convert to a packet object

Client:

Attribute	Description
client_id: Int	A UUID of the client
socket: SocketHandler	Client socket handler
cout: StreamOutputObject	Object used to stream text to the user's screen
cin: StreamInputObject	Object used to read text from the user

Method	Description
handlePacket(Packet): void	Handle a packet sent from the server
ServerConnect(): void	Connect to the server

Gameboard:

Attribute	Description
board: LinkedList<Room>	

Method	Description
Board()	constructor
Move(Player, Room): Boolean	Move a player to another room
getValidMoves(Player): List<Room>	Get all valid rooms a player can move too
getStartingRoom(Player): Room	Get the starting room of a player
getRoomOccupants(String): List<Player>	Get the occupants of a room by its name
getConnectingRooms(String): List<Room>	Get the connecting rooms of a room by its name

Room:

Attribute	Description
room_name: String	Name of the room
is_hallway: Boolean	Is this room a hallway
occupant_players: List<Players>	The players in a room

Method	Description
Room()	Constructor
toString()	get the room name
getOccupants(): List<Player>	Get players in the room

DeckHelper:

Attribute	Description
crime_scene_cards: Deck	A deck of cards representing all crime scenes
weapon_cards: Deck	A deck of cards representing all murder weapons
player_cards: Deck	A deck of cards representing all suspects

Method	Description
DeckHelper()	Constructor
getSecretCards(): List<Card>	Shuffle each deck and get the secret cards
dealHands(Int): Map<int, Deck>	Divy cards into multiple hands

Deck:

Attribute	Description
cards: List<Card>	

Method	Description
Deck(List<Cards>)	Constructor
shuffleDeck(): void	Shuffle the deck of cards in place
pop(): Card	Get the card at the top of the deck
push(Card): void	Add a card to the deck
extend(Deck): void	Add all elements from another deck into this deck

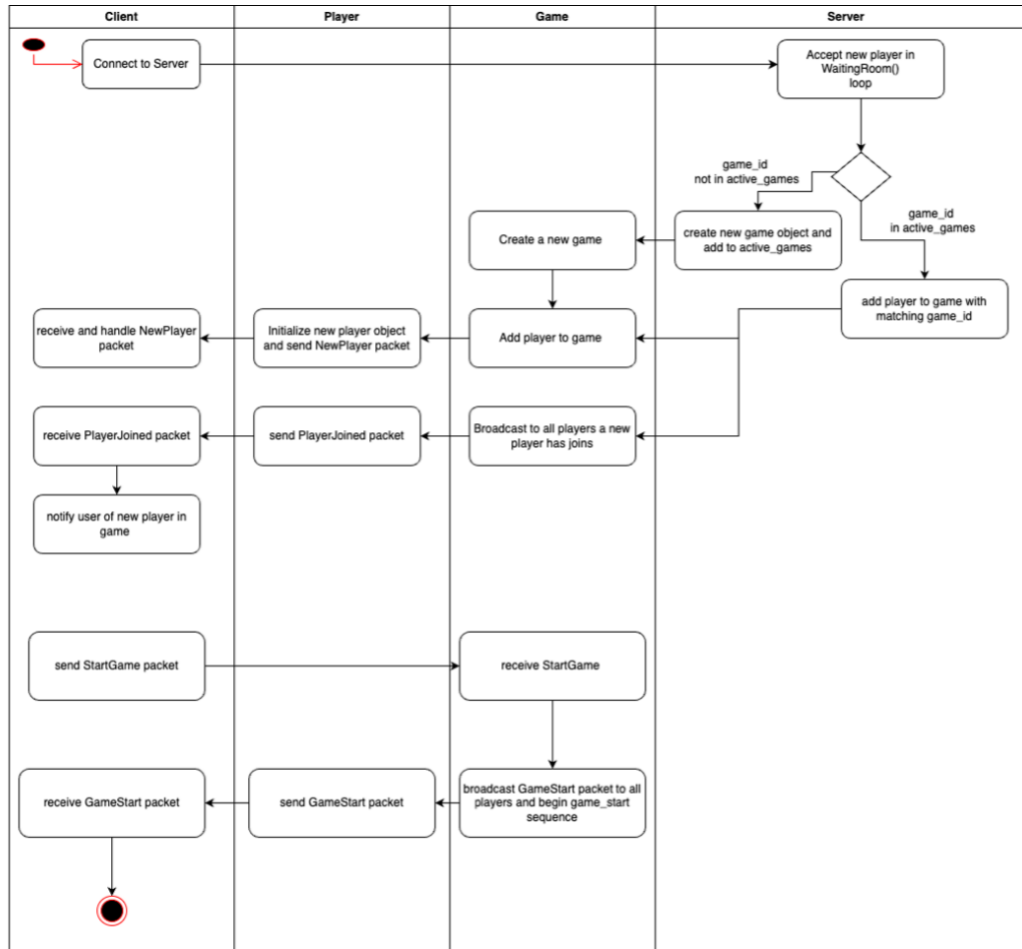
Card:

Attribute	Description
name: String	Card name
type: Enum[Room, Weapon, Character]	Card type

Method	Description
Card(String, Enum)	Constructor
toString(): String	Get the name of the card
getType(): Enum	Get the type of the card

System Operations

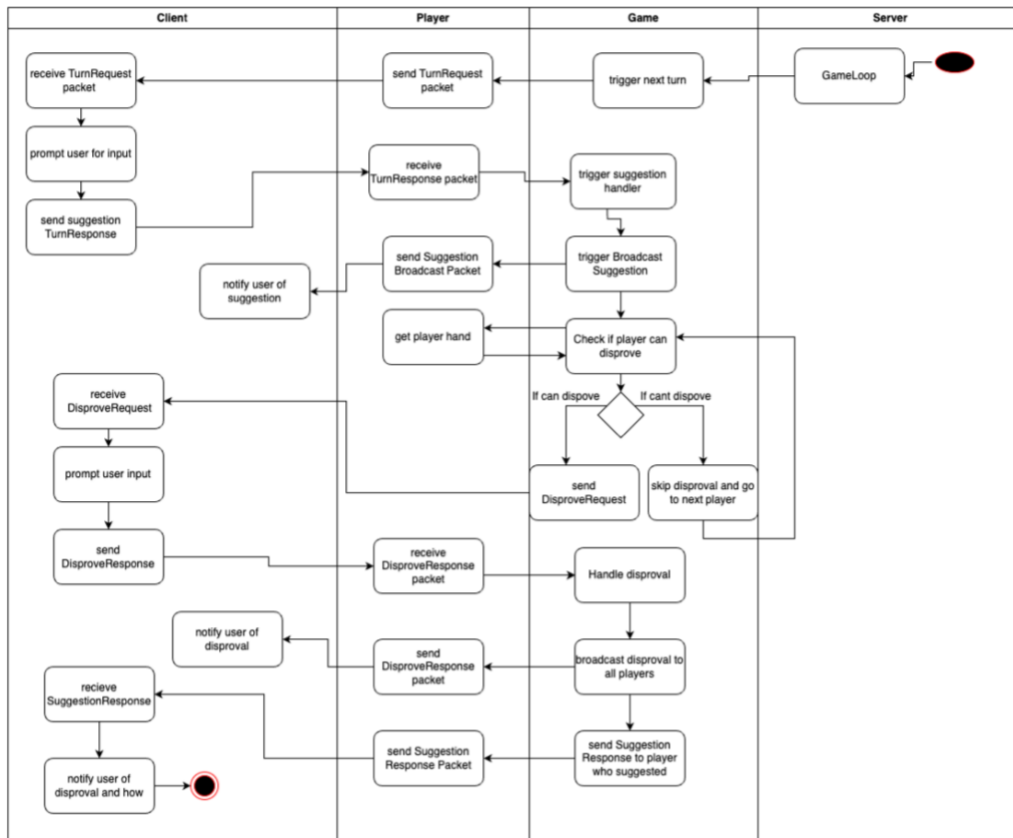
Use Case 1: Connect to Game



In this use case, a player connects to the game by establishing a connection with the server. The server authenticates the player and assigns them to an available game instance or lobby, preparing the player for gameplay.

- **Key Steps:**
 - Client initiates the connection to the game server.
 - Server authenticates the connection and verifies player details.
 - Player is assigned to a game instance or lobby.
 - The system prepares the player for the upcoming game by syncing game state and ensuring proper setup.

Use Case 2: Suggestion Loop

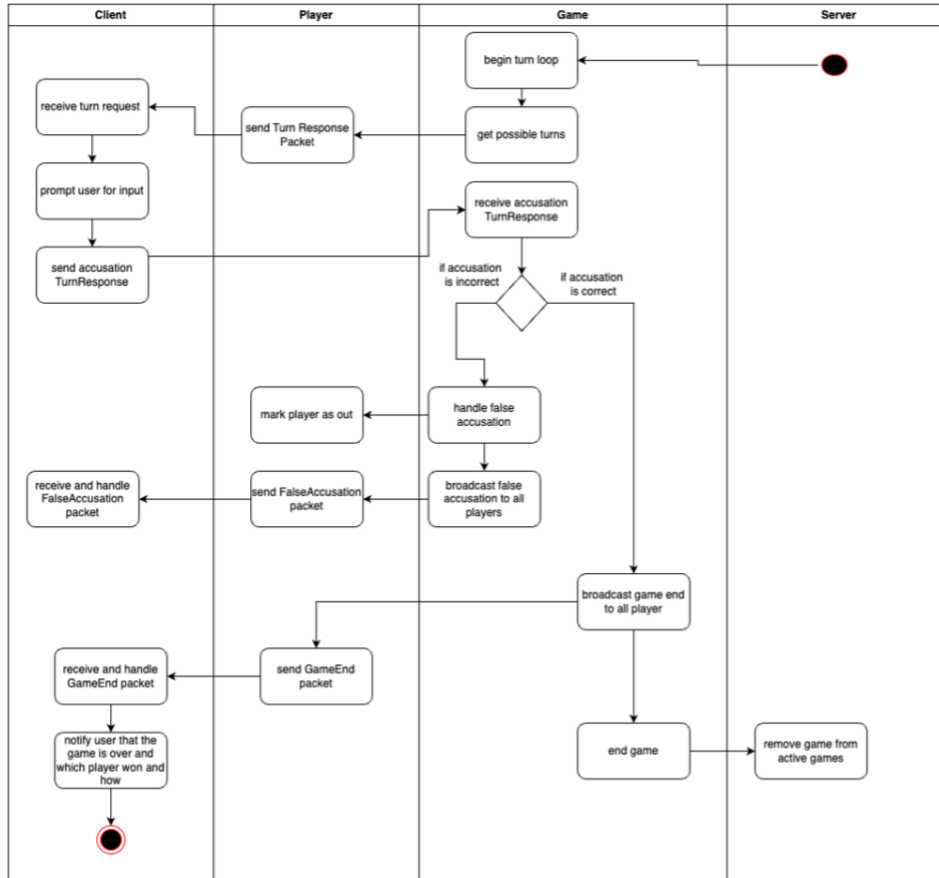


In this use case, the player makes a suggestion to identify the suspect and weapon associated with the crime, based on their current location in a room. The suggestion is broadcast to all players, and other players can try to disprove the suggestion by showing relevant cards.

- **Key Steps:**

- Player selects a suspect and weapon.
- System checks for disproving actions from other players.
- Suggestion waits for a response.

Use Case 3: Make an Accusation

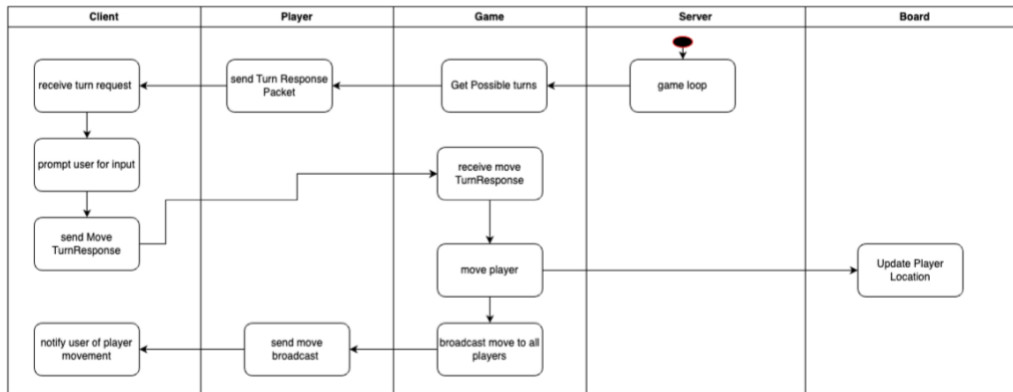


This use case covers the situation where a player attempts to make a final accusation, declaring a suspect, weapon, and room. If correct, the player wins the game. If incorrect, the player is disqualified from further actions but can still observe the game.

- **Key Steps:**

- Player selects a suspect, weapon, and room for the accusation.
- System checks the accuracy of the accusation.
- Game ends if correct; player is disqualified if incorrect.

Use Case 4: Move to a Room/Hallway



In this use case, the player moves from their current location to an adjacent room via a hallway. The system ensures that movement is valid and that hallways are not blocked by other players.

- **Key Steps:**

- Player selects an adjacent room.
- System checks for valid movement.
- Game state is updated with the new player location.